

Forward Propagation in Python (From Scratch)

The goal of this exercise is to build intuition about what TensorFlow is doing "under the hood." By implementing the calculations manually, you can understand the mechanics of a neural network.

Note: This implementation will use **1D NumPy arrays** (e.g., `np.array([...])`) for vectors and parameters, unlike the 2D matrices (e.g., `np.array([[...]])`) expected by TensorFlow's `Sequential` model. This simplifies the from-scratch code.

Coffee Roasting Example Recap

- **Input ($\vec{x}$):** 2 features
 - **Layer 1 ($\vec{a}^{[1]}$):** 3 neurons
 - **Layer 2 ($\vec{a}^{[2]}$):** 1 neuron (output)
-

Implementation Steps 🧑🏻💻

We will compute the activation for each neuron, one by one.

1. Compute Layer 1 Activations

- **Input:** `x` (a 1D NumPy array with 2 features)
- **Neuron 1 ($a_1^{[1]}$):**
 1. Define parameters: `w1_1` (vector), `b1_1` (scalar)
 2. Compute `z`: `z1_1 = np.dot(w1_1, x) + b1_1`
 3. Apply activation: `a1_1 = sigmoid(z1_1)`
- **Neuron 2 ($a_2^{[1]}$):**
 1. Define parameters: `w1_2`, `b1_2`
 2. Compute `z`: `z1_2 = np.dot(w1_2, x) + b1_2`
 3. Apply activation: `a1_2 = sigmoid(z1_2)`
- **Neuron 3 ($a_3^{[1]}$):**
 1. Define parameters: `w1_3`, `b1_3`
 2. Compute `z`: `z1_3 = np.dot(w1_3, x) + b1_3`
 3. Apply activation: `a1_3 = sigmoid(z1_3)`
- **Collect Activations:**
 - Group the individual neuron outputs into a single vector, which is the output of

Layer 1.

- `a1 = np.array([a1_1, a1_2, a1_3])`

2. Compute Layer 2 (Output Layer) Activation

- **Input:** `a1` (the 1D NumPy array with 3 activations from Layer 1)
- **Neuron 1 ($a_1^{[2]}$):**
 1. Define parameters: `w2_1` (vector), `b2_1` (scalar)
 2. Compute z : `z2_1 = np.dot(w2_1, a1) + b2_1`
 3. Apply activation: `a2_1 = sigmoid(z2_1)`
- **Final Prediction:** The value `a2_1` is the final output (probability) of the network.

This manual, neuron-by-neuron process is exactly what a framework like TensorFlow does for you, but in a highly optimized, vectorized way.

A General Implementation of Forward Propagation

Instead of hard-coding the computation for every single neuron, we can create a general-purpose function, `dense`, to compute the output of an entire layer.

The dense Function 🧑

The goal is to create a function that computes the activations for one layer.

- **Function definition:** `def dense(a_in, w, b)`
 - **Inputs:**
 - `a_in`: The activation vector from the **previous** layer (for the first layer, this would be the input x).
 - `w`: The weight parameters for the **current** layer.
 - `b`: The bias parameters for the **current** layer.
 - **Returns:**
 - `a_out`: The activation vector of the **current** layer.
-

Parameter Organization

To make this function work, we must organize our parameters into matrices and vectors.

- **w (Weight Matrix):** The weight vectors for each neuron in the layer are stacked as **columns** to form a matrix.
 - If Layer 1 has 3 neurons, `w1` would be a matrix where the 1st column is $\vec{w}_1^{[1]}$, the 2nd column is $\vec{w}_2^{[1]}$, and the 3rd column is $\vec{w}_3^{[1]}$.
 - **b (Bias Vector):** The bias values for each neuron are stored in a 1D array.
 - `b1 = [b^{[1]}_1, b^{[1]}_2, b^{[1]}_3]`
-

Implementation Logic of the dense Function

The function iterates through each neuron (each column of `w`) and performs the standard logistic regression calculation.

```
1 # Pseudocode for the dense function
2 def dense(a_in, w, b):
3
4     # Get the number of neurons (units) from the number of columns in w
5     units = w.shape[1]
6
```

```

7      # Initialize the output activation vector
8      a_out = np.zeros(units)
9
10     # Loop over each neuron
11     for j in range(units):
12
13         # Get the weight vector for the j-th neuron (the j-th column)
14         w = W[:, j]
15
16         # Calculate  $z = w_j \cdot a_{in} + b_j$ 
17         z = np.dot(a_in, w) + b[j]
18
19         # Calculate the activation for the j-th neuron
20         a_out[j] = g(z) # g is the sigmoid function
21
22     return a_out

```

Building the Full Network

With this dense function, you can build a full neural network by "stringing together" the layers sequentially, passing the output of one layer as the input to the next.

```

1  # Example of a 4-layer network
2  def my_network(x, w1, b1, w2, b2, w3, b3, w4, b4):
3
4      a1 = dense(x, w1, b1)
5      a2 = dense(a1, w2, b2)
6      a3 = dense(a2, w3, b3)
7      a4 = dense(a3, w4, b4)
8
9      f_x = a4
10     return f_x

```

Why Implement This Manually?

- Although in practice you will use libraries like TensorFlow, understanding what happens "under the hood" is critical.
- This knowledge is essential for **debugging** your models when they don't work, run slowly, or produce unexpected results.